

Unión Europea

Fondo Social Europeo

El FSE invierte en tu futuro

Laravel

Servicios Rest

Pepe Devesa





Continguts

1 Creación de servicios REST **3**

2 Definiendo los controladores de API **3**

2.1 Estableciendo las rutas 5

3 Servicios GET (index y show) **5**

3.1 Más sobre el formato JSON y la respuesta 7

4 Resto de servicios **8**

4.1 Respuestas de error 9

1 Creación de servicios REST

Veamos ahora qué pasos dar para construir una API REST en Laravel que dé soporte a las operaciones básicas sobre una o varias entidades. Emplearemos para ello los denominados `controladores de API`, que comentamos brevemente en unidades anteriores, al hablar de controladores, y que proporcionan un conjunto de funciones ya definidas para dar soporte a cada uno de estos comandos.

2 Definiendo los controladores de API

Para proporcionar una API REST a los clientes que lo requieran, necesitamos definir un controlador (o controladores) orientados a ofrecer estos servicios REST. Estos controladores en Laravel se denominan de tipo api, como vimos en sesiones previas. Normalmente se definirá un controlador API por cada uno de los modelos a los que necesitemos acceder. Vamos a crear uno de prueba para ofrecer una API REST sobre los libros de nuestra aplicación de biblioteca. Existen diferentes formas de ejecutar el comando de creación del controlador de API. Aquí vamos a mostrar quizá una de las más útiles:

```
php artisan make:controller Api/LibroController --api --model=Libro
```

Esto creará el controlador en la carpeta `App\Http\Controllers\Api` con una serie de funciones ya predefinidas. No es obligatorio ubicarlo en esa subcarpeta, obviamente, pero esto nos servirá para separar los controladores de API del resto. Esta será la apariencia del controlador generado:

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Libro;
use Illuminate\Http\Request;

class LibroController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        //
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
```

```
    {
        //
    }

    /**
     * Display the specified resource.
     */
    public function show(Libro $libro)
    {
        //
    }

    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, Libro $libro)
    {
        //
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(Libro $libro)
    {
        //
    }
}
```

Observemos que se incorpora automáticamente la cláusula `use` para cargar el modelo asociado, que hemos indicado en el parámetro `--model`. Además, los métodos `show`, `update` y `destroy` ya vienen con un parámetro de tipo `Libro` que facilitará mucho algunas tareas.

Cada una de las funciones del nuevo controlador creado se asocia a uno de los métodos REST comentados anteriormente:

- `index` se asociaría con una operación **GET** de listado general, para obtener todos los registros (de libros, en este caso)
- `store` se asociaría con una operación **POST**, para almacenar los datos que lleguen en la petición (como un nuevo libro, en nuestro caso)
- `show` se asociaría con una operación **GET** para obtener el registro asociado a un identificador concreto
- `update` se asociaría con una operación **PUT**, para actualizar los datos del registro asociado a un identificador concreto
- `destroy` se asociaría con una operación **DELETE**, para eliminar los datos del registro asociado

a un identificador concreto.

2.1 Estableciendo las rutas

Una vez tenemos el controlador API creado, vamos a definir las rutas asociadas a cada método del controlador. Si recordamos de sesiones anteriores, podíamos emplear el método `Route::resource` en el archivo `routes/web.php` para establecer de golpe todas las rutas asociadas a un controlador de recursos. **De forma análoga**, podemos emplear el método `Route::apiResource` en el archivo `routes/api.php` para establecer automáticamente todas las rutas de un controlador de API. Añadimos esta línea en dicho archivo routes/api.php:

```
use App\Http\Controllers\Api\LibroController;
...
Route::apiResource('libros', LibroController::class);
```

Las rutas de API (aquellas definidas en el archivo `routes/api.php`) por defecto tienen un prefijo `api` , tal y como se establece en el *provider* `RouteServiceProvider` . Por tanto, hemos definido una ruta general `api/libros` , de forma que todas las subrutas que se deriven de ella llevarán a uno u otro método del controlador de API de libros.

Podemos comprobar qué rutas hay activas con este comando:

```
php artisan route:list
```

```
GET|HEAD   api/libros ..........libros.index Api\LibroController@index
POST       api/libros ..........libros.store Api\LibroController@store
GET|HEAD   api/libros/{libro} ..libros.show  Api\LibroController@show
PUT|PATCH  api/libros/{libro} ..libros.update Api\LibroController@update
DELETE  api/libros/{libro} ..libros.destroy Api\LibroController@destroy
```

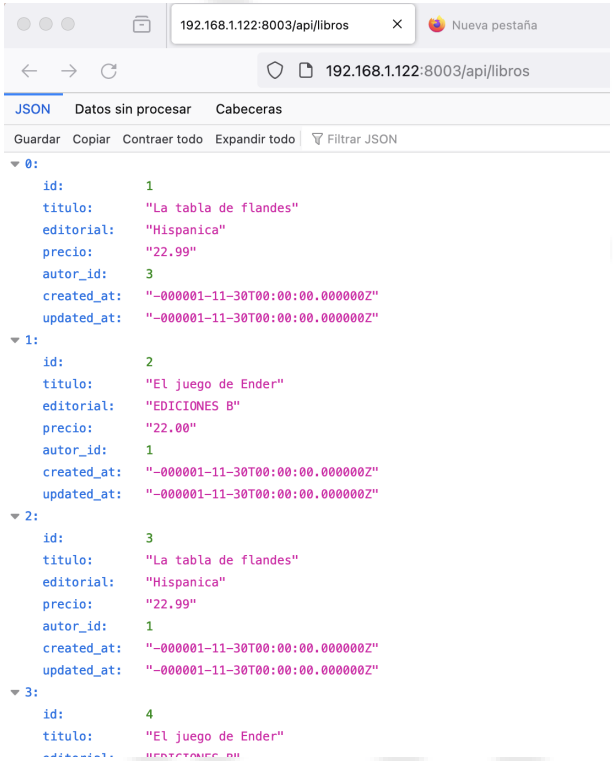
3 Servicios GET (index y show)

Vamos a empezar por definir el método `index`. En este caso, vamos a obtener el conjunto de libros de la base de datos y devolverlo tal cual:

```
public function index() {

    $libros = Libro::get();
    return $libros;
}
```

Si accedemos a la ruta `api/libros` desde el navegador, se activará el método index que acabamos de implementar, y recibiremos los libros de la base de datos, directamente en formato JSON.

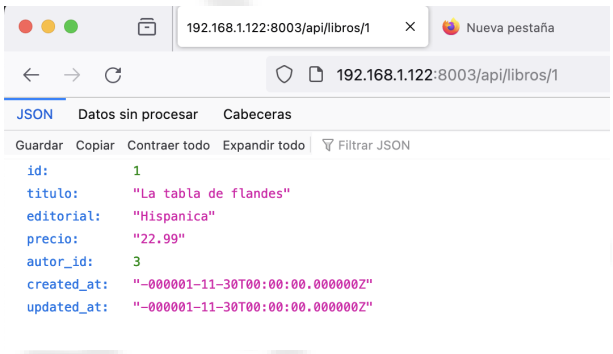


(a) Formato JSON

De una forma similar, podríamos implementar y probar el método show , para mostrar los datos de un libro en particular:

```
public function show(Libro $libro)
{
 return $libro;
}
```

En este caso, si accedemos a la URI `api/libros/1` , obtendremos la información del libro con id = 1. Notar que Laravel se encarga automáticamente de buscar el libro por nosotros (hacer la correspondiente operación find para el id proporcionado). Es lo que se conoce como *enlace implícito*, y es algo que también está disponible en los controladores web normales, siempre que los asociemos correctamente con el modelovinculado. Esto se hace automáticamente si creamos el controlador junto con el modelo, si usamos el parámetro `--model` para asociarlo, como hemos hecho aquí.



(b) Show JSON

3.1 Más sobre el formato JSON y la respuesta

Laravel se encarga de transformar directamente los registros obtenidos a formato JSON cuando los enviamos mediante **return**, por lo que, en principio, no tenemos por qué preocuparnos de este proceso. Sin embargo, de este modo se escapan algunas cosas a nuestro control. Por ejemplo, y sobre todo, no podemos especificar el código de estado de la respuesta, que por defecto es 200 si todo ha ido correctamente. Además, tampoco podemos controlar qué información enviar del objeto en cuestión.

- Añadir cláusulas `hidden` en los modelos correspondientes, para indicar que esa información no debe ser enviada en ningún caso en ninguna parte de la aplicación. Es lo que ocurre, por ejemplo, con el campo password del modelo de Usuario :

```
protected $hidden = ['password'];
```

- Definir a mano un array con los campos a enviar en el método del controlador. En el caso de la ficha del libro anterior, si sólo queremos enviar el título y la editorial, podríamos hacer algo así:

```
public function show(Libro $libro)
{
    return [
    'titulo' => $libro->titulo,
    'editorial' => $libro->editorial
    ];
}
```

Por otra parte, si queremos añadir o modificar más información en la respuesta, como el código de estado, la estructura anterior no nos sirve, ya que siempre se va a enviar un código 200. Para esto, es conveniente emplear el método `response()->json(...)`, que permite especificar como primer parámetro los datos a enviar, y como segundo parámetro el código de estado. Los métodos anteriores

quedarían así, enviando un código 200 como respuesta (aunque si se omite el segundo parámetro, se asume que es 200):

```
public function index()
{
    $libros = Libro::get();
    return response()->json($libros, 200);;
}
...
public function show(Libro $libro)
{
    return response()->json($libro, 200);
}
```

4 Resto de servicios

Veamos ahora cómo implementar el resto de servicios (POST, PUT y DELETE). En el caso de la inserción (**POST**), deberemos recibir en la petición los datos del objeto a insertar (un libro, en nuestro ejemplo). Igual que los datos del servidor al cliente se envían en formato JSON, es de esperar en aplicaciones que siguen la arquitectura REST que los datos del cliente al servidor también se envíen en formato JSON. Nuestro método `store`, asociado al servicio POST, podría quedar de este modo (devolvemos el código de estado 201, que se utiliza cuando se han insertado elementos nuevos):

```
public function store(Request $request)
{
    $libro = new Libro();
    $libro->titulo = $request->titulo;
    $libro->editorial = $request->editorial;
    $libro->precio = $request->precio;
    $libro->autor()->associate(Autor::findOrFail($request->autor_id));
    $libro->save();
    return response()->json($libro, 201);
}
```

De forma similar implementaríamos el servicio **PUT**, a través del método `update`. En este caso devolvemos un código de estado 200:

```
public function update(Request $request, Libro $libro)
{
    $libro->titulo = $request->titulo;
    $libro->editorial = $request->editorial;
    $libro->precio = $request->precio;
    $libro->autor()->associate(Autor::findOrFail($request->autor_id));
    $libro->save();
    return response()->json($libro);
}
```

Finalmente, para el servicio **DELETE**, debemos implementar el método `destroy` , que podría quedar así:

```
public function destroy(Libro $libro)
{
    $libro->delete();
    return response()->json(null, 204);
}
```

4.1 Respuestas de error

Por otra parte, debemos asegurarnos de que cualquier error que se produzca en la parte de la API devuelva un cotenido en formato JSON, y no una página web. Por ejemplo, si solicitamos ver la ficha de un libro cuyo id no existe, no debería devolvernos una página de error 404, sino un código de estado 404 con un mensaje de error en formato JSON. El método a modificar se llama `register` , dentro de la misma clase `App\Exceptions\Handler.php`. Lo podemos dejar de este modo para hacer algo equivalente a lo anterior:

```
 public function register(): void
    {
        $this->reportable(function (Throwable $e) {
            //
            if (request()->is('api*'))
                {
                if ($exception instanceof ModelNotFoundException)
                    return response()->json(['error' => 'Recurso no
                       encontrado'],404);
                else if ($exception instanceof ValidationException)
                    return response()->json(['error' => 'Datos no válidos'
                       ],400);
                else if (isset($exception))
                    return response()->json(['error' => 'Error: ' .
                       $exception->getMessage()], 500);
                }
        });
    }
```